Reproducing WLasso on breast cancer gene expression data

Mariana Garcia Mejia

October 22, 2026

0.1 Reproducing WLasso on breast cancer gene expression data (GSE2990)

0.1.1 Data and preprocessing

We use data from the Gene Expression Omnibus Gene Expression Omnibus with GEO Accession GSE2990 as used by the authors (Zhu et al., 2020). The data contains $n = 189$ tumours and 22,283 probes (HG-U133A array). The values range from 2.66 to 15.13, which is consistent with RMA-normalised data.

```
# Load the gene data from Gene Expression Omnibus
expr <- read.delim("GSE2990_series_matrix.txt",
                   comment.char = "!", row.names = 1)
dim(expr)
```

```
[1] 22283   189
```

```
# Values appear to be normalized (between 2 and 15).
range(as.matrix(expr), na.rm = TRUE)
```

```
[1]  2.656036 15.129603
```

As in Zhu et al. (2021), we kept probes with IQR ≥ 1.5 (1,155 probes) and removed probes without a gene annotation (49 removed, 1,106 left).

```
# First cleaning step: remove probes with IQR < 1.5
iqr_values <- apply(expr, 1, IQR)
```

```r
# summary(iqr_values)
keep <- iqr_values >= 1.5
# sum(keep)
expr_filt <- expr[keep, ]
# dim(expr_filt)

# Second step: Remove genes without gene annotation
# BiocManager::install("hgu133a.db")
library(hgu133a.db)

probe_ids <- rownames(expr_filt)
symbols <- mapIds(hgu133a.db,
                  keys = probe_ids,
                  column = "SYMBOL",
                  keytype = "PROBEID",
                  multiVals = "first")

# sum(is.na(symbols))

# Remove
has_symbol <- !is.na(symbols)
expr_ann <- expr_filt[has_symbol, ]
symbols_ann <- symbols[has_symbol]

dim(expr_ann)
```

```
[1] 1106  189
```

The ESR1 probe is a gene associated with breast cancer. We use it as the response y and the remaining 1,105 probes as predictors X. As with regular As is convention in Lasso, we standardised each predictor. We also centred y, because the criterion has no intercept. The paper reports $p = 1{,}111$, so our probe set differs slightly, $p = 1{,}106$.

```r
# Use ESR1 gene as the gene to predict.
all_symbols <- mapIds(hgu133a.db,
                      keys = rownames(expr),
                      column = "SYMBOL",
                      keytype = "PROBEID",
                      multiVals = "first")

esr1_probes <- names(all_symbols)[all_symbols %in% "ESR1"]
```

```r
# esr1_probes
# esr1_probes %in% rownames(expr_ann)

# Pull the ESR Variable left after IQR + Annotation filter and use as y
esr1_probe <- esr1_probes[1]
# esr1_probe
y <- as.numeric(expr_ann[esr1_probe, ])
expr_X <- expr_ann[rownames(expr_ann) != esr1_probe, ]

# length(y)
# dim(expr_X)

# Standardize X.
X <- scale(t(expr_X))
# dim(X)
# round(colMeans(X)[1:3], 6)
# apply(X, 2, sd)[1:3]

# Mean-center Y.
y_c <- y - mean(y)
# mean(y_c) # should be close to zero.
```

0.1.2 What the gene annotation does

Each column $j \in \{1, \dots, p\}$ of X is a probe, and the array itself only knows probe IDs. An annotation is a lookup map g from probe indices to gene symbols (or to NA when the probe has no known gene). We used the Bioconductor package `hgu133a.db` for this array. It never enters the estimation of β, only the steps before and after:

- **Before:** we drop probes with $g(j) = \text{NA}$, and we find the probe j^* with $g(j^*) = \text{ESR1}$ to build $y = X_{j^*}$ and take it out of the predictors.
- **After:** the method returns a support $\widehat{S} = \{j : \hat{\beta}_j \neq 0\}$ of probes. The map turns it into a list of genes $g(\widehat{S})$. Several probes can map to the same gene, so $|g(\widehat{S})| \leq |\widehat{S}|$.

0.1.3 Method

The model is $y = X\beta + \epsilon$ with $\text{Cor}(X) = \Sigma$. WLasso whitens the predictors with $\tilde{X} = X\Sigma^{-1/2}$, so that $y = \tilde{X}\tilde{\beta} + \epsilon$ with $\tilde{\beta} = \Sigma^{1/2}\beta$, and the error term is unchanged. It then minimises the generalised Lasso criterion

$$\|y - \tilde{X}\tilde{\beta}\|_2^2 + \lambda\|\Sigma^{-1/2}\tilde{\beta}\|_1,$$

and thresholds the estimates to the top K (for $\tilde{\beta}$) and top M (for β) coefficients, with K and M chosen by an MSE-ratio rule controlled by γ. Σ was estimated with the two-block structure of Section 2.5: $(\hat{\alpha}_1, \hat{\alpha}_2, \hat{\alpha}_3) = (0.19, 0.27, 0.55)$, against $(0.17, 0.21, 0.52)$ in the paper.

0.1.4 A problem with the package and our patch

With the `WLasso` package (v1.0), `Whitening_Lasso()` returned an empty model on these data (the lowest MSE was at the first, all-zero step), although a plain Lasso on the same data selected 62 probes. The package works on its own example data and on simulated data of our size ($n = 189$, $p = 1105$). Reading the source, in the step that chooses M, the MSE is computed from the coefficients sorted by absolute value, not from the coefficients in their original order, so each coefficient multiplies the wrong column of X. We think this is a bug that does little harm when all correlations are positive, as in the paper's simulations. We changed that one argument in a copy of the function (not an official fix, and not confirmed with the authors), and results below come from that modified copy.

```
# Install WLasso package
# install.packages("WLasso")
library(WLasso)
# help(package = "WLasso")
```

```
# Step 1: Sigma Estimation
Sigma_est <- Sigma_Estimation(X)
round(Sigma_est$alpha, 2) # paper reports 0.17, 0.21 and 0.52, we get slightly different
```

```
[1] 0.19 0.27 0.55
```

```
# We get: 0.19 0.27 0.55

# Number of probes the clustering put into one of the two blocks
# We get 181 out of 1105
length(Sigma_est$group_act)
```

```
[1] 181
```

```
# Step 2: Model transformation
Sigma_estmat <- Sigma_est$mat
```

```r
# Fix bug on the paper
src <- paste(deparse(WLasso::Whitening_Lasso), collapse = "\n")
src2 <- gsub("x\\s*=\\s*beta_sort,\\s*sorted_vect\\s*=\\s*beta_sort",
             "x = beta_final0, sorted_vect = beta_sort", src)

# identical(src, src2)   # we want FALSE this time

WL_fixed <- eval(parse(text = src2))
environment(WL_fixed) <- asNamespace("WLasso")
# grep("beta_sort", strsplit(src, "\n")[[1]], value = TRUE)
# grep("beta_interm", strsplit(src2, "\n")[[1]], value = TRUE)
```

```r
# Fit the model
# mod_fix <- WL_fixed(X = X, Y = y_c, Sigma = Sigma_estmat,
                      # gamma = 0.95, maxsteps = 500)

# range(mod_fix$mse)
# sum(mod_fix$beta.min != 0)
```

```r
# We also fit regular Lasso for comparison:
#install.packages("glmnet")
library(glmnet)
```

```
Warning: package 'glmnet' was built under R version 4.3.3
```

```
Loading required package: Matrix
```

```

Attaching package: 'Matrix'
```

```
The following object is masked from 'package:S4Vectors':

    expand
```

```
Loaded glmnet 4.1-9
```

```r
cvfit <- cv.glmnet(X, y_c)
sum(coef(cvfit, s = "lambda.min")[-1] != 0)
```

```
[1] 60
```

```
# returns 61 coefficients!!!!
```

0.1.5 Results

Method	Probes	Genes
WLasso ($\gamma = 0.95$, patched)	59	56
Lasso (cross-validation)	62	59

The paper reports 63 and 66 genes. The two methods share 38 genes in our run, against 8 in the paper. NAT1 and GATA3, both discussed in the paper, are selected by both methods. Varying γ changes the size of the selection (42, 56 and 72 genes for $\gamma = 0.90, 0.95, 0.99$), but 36 genes are selected at all three values. Some genes discussed in the paper are unstable: TOP2A is selected at 0.90 and 0.99 but not at 0.95, and CXCL12 only at 0.99.

0.1.6 Limitations

The true set of relevant genes is unknown, so this is an illustration, as in the paper, and cannot show that one method is better. We only checked stability with respect to γ, not $\hat{\Sigma}$ or the sample. The results rely on the patched function and the current annotation version.

Reference. Zhu, W., Lévy-Leduc, C. and Ternès, N. (2021). A variable selection approach for highly correlated predictors in high-dimensional genomic data. *Bioinformatics*, 37(16), 2238–2244.